SOC L1 LAB · INC-001

Reproduction Guide — INC-001 Lab

Rebuilding the lab from a bare host to a working detection pipeline

INCIDENT	INC-001 — SSH brute force to persistence
REPORT DATE	2026-10-31
CLASSIFICATION	Lab exercise — sanitized for publication
ENVIRONMENT	Wazuh 4.14.7 · Debian 12 · QEMU/KVM
SEVERITY	High

Independent portfolio lab. No production systems, client data, or real credentials.

Contents

0. Prerequisites
1. Part A — Wazuh stack
2. Part B — Victim VM
3. Part C — Custom rule
4. Part D — Network sensor (optional for INC-001)
5. Part E — Baseline snapshot
6. Part F — Run the incident
7. Part G — Verify and extract
8. Part H — Clean reset
9. Troubleshooting
10. Reproduction checklist
11. Teardown

This document rebuilds the environment from a bare host and reproduces the detection pipeline. It is written to be followed in order. Commands assume a Debian/Arch-family Linux host with Docker, QEMU, and `hydra`.

Time: 60–90 minutes on a first run, ~25 minutes if the base image is cached.

Result: a Wazuh stack, one Debian 12 endpoint with agent, auditd, FIM and Suricata, a custom persistence rule, and a repeatable attack that produces the INC-001 evidence.

---

0. Prerequisites

0.1 Hardware

Resource	Minimum	Used here
RAM	6 GB	7.5 GB
Disk	15 GB free	~8 GB used by images, VM, and logs
CPU	4 cores	12 cores
Virtualisation	KVM enabled	`/dev/kvm` present

The VM runs with 1.5 GB. The Wazuh stack uses ~3 GB. Do not run the full ET Open ruleset in the VM; it needs ~1.1 GB and will be OOM-killed (see §11).

0.2 Software

```
docker --version          # 24+ (29.7.2 used here)
docker compose version    # v2+
qemu-system-x86_64 --version
qemu-img --version
hydra -h | head -1
curl --version | head -1
jq --version
```

0.3 Network

- Internet access for image download, apt, and the Wazuh packages.
- No inbound exposure. All published Docker ports bind to `127.0.0.1` / `::1`.
- The VM uses QEMU user networking; it has no route to your LAN.

0.4 Layout

```
export LAB="$HOME/Documents/Eterna/soc-l1-lab"
cd "$LAB"
```

```
soc-l1-lab/
├── lab/wazuh/           # stack
├── lab/vm/              # VM launch, seed, provisioning
├── lab/suricata/        # sensor notes
├── rules/               # custom rules, mounted into the manager
├── docs/evidence/       # INC-001 output
└── tools/md2pdf.py
```

---

1. Part A — Wazuh stack

1.1 Docker access

Either add your user to the `docker` group (log out/in afterwards) or use the socket ACL already present on this host.

```
sudo usermod -aG docker "$USER"
# or, for the current shell only:
newgrp docker
```

Verify:

```
docker info --format '{{.ServerVersion}}'
```

1.2 Certificate directory

The upstream checkout may have a root-owned certificate directory from a previous run:

```
ls -ld "$HOME/wazuh-docker/single-node/config/wazuh_indexer_ssl_certs"
# dr-x------ root root
```

The container UIDs are 1000 (indexer) and 999 (manager). If the files are already owned by those UIDs, leave them alone. If the directory itself blocks traversal, run the generator again or fix ownership **as root** before starting.

Do not blanket `chown` the certificate files; it breaks the container's expected UIDs.

1.3 Start

```
cd "$LAB/lab/wazuh"
./setup.sh
```

`setup.sh` pins `v4.14.7`, generates certificates if missing, starts the three containers, and waits for the dashboard.

Expected:

```
single-node-wazuh.dashboard-1   running
single-node-wazuh.indexer-1     running
single-node-wazuh.manager-1     running
dashboard is up
```

1.4 Bind ports to localhost

The upstream compose publishes on `0.0.0.0`. For a lab, restrict it:

```
ports:
  - "127.0.0.1:443:5601"
  - "[::1]:443:5601"      # makes https://localhost work when it resolves to ::1
  - "127.0.0.1:1514:1514"
  - "127.0.0.1:1515:1515"
  - "127.0.0.1:514:514/udp"
  - "127.0.0.1:55000:55000"
  - "127.0.0.1:9200:9200"
```

Recreate:

```
cd "$HOME/wazuh-docker/single-node"
docker compose up -d
```

Verify:

```
curl -ks -o /dev/null -w '%{http_code}\n' https://localhost          # 302
curl -ks -u 'wazuh-wui:MyS3cr37P450r.*-' \
  -o /dev/null -w '%{http_code}\n' \
  https://localhost:55000/security/user/authenticate                 # 200
```

Dashboard: `https://localhost`, `admin` / `SecretPassword`. The certificate is self-signed and issued for `wazuh.dashboard`, so the browser shows a name mismatch. Advanced → Proceed once.

1.5 Reduce startup noise (optional)

The manager logs a vulnerability-scanner feed error:

```
ERROR: Error updating feed: [json.exception.out_of_range.403] key 'other' not found
```

It is an upstream feed schema issue and does not affect alerting. Vulnerability detection is out of scope for INC-001.

---

2. Part B — Victim VM

2.1 Download and first boot

```
cd "$LAB/lab/vm"
./launch.sh
```

What it does:

1. Downloads the Debian 12 genericcloud image (~350 MB) to `images/`.
2. Generates an SSH key at `keys/id_ed25519`.
3. Renders `seed/user-data` from the template and starts a seed HTTP server on `127.0.0.1:8000`.
4. Creates a 20 GB qcow2 overlay.
5. Boots QEMU with user networking, port forwards 2222→22 and 8080→80, and a monitor socket.

Expected:

```
VM started.
  ssh     : ssh -i keys/id_ed25519 -p 2222 analyst@127.0.0.1
  http    : http://127.0.0.1:8080
```

2.2 Wait for cloud-init

The first boot installs `nginx`, `auditd`, `jq`, and Suricata from backports. This takes 3–8 minutes.

```
ssh -i keys/id_ed25519 -p 2222 \
  -o StrictHostKeyChecking=no analyst@127.0.0.1 \
  'cloud-init status --wait; cloud-init status'
```

Expected:

```
status: done
```

If it reports `error`, jump to §11, item 2.

2.3 Enrol the agent

```
cd "$LAB/lab/vm"
scp -i keys/id_ed25519 -P 2222 -o StrictHostKeyChecking=no \
  provision-agent.sh analyst@127.0.0.1:/tmp/provision-agent.sh
ssh -i keys/id_ed25519 -p 2222 -o StrictHostKeyChecking=no \
  analyst@127.0.0.1 'sudo bash /tmp/provision-agent.sh'
```

The script:

- installs `wazuh-agent` 4.14.7 with `WAZUH_MANAGER=10.0.2.2`;

- loads audit rules using Wazuh's expected keys (`audit-wazuh-w` , `audit-wazuh-c`);
- narrows Suricata to scan/web/DNS groups;
- installs a `tail | jq` filter that writes `/var/log/suricata/wazuh.json` ;
- adds a `<localfile>` for that JSON file;
- restarts the agent.

Verify on the manager:

```
docker exec single-node-wazuh.manager-1 /var/ossec/bin/agent_control -l
# ID: 001, Name: soc-victim, IP: any, Active
```

2.4 FIM on `/home`

The stock agent config scans every 12 hours. Add realtime coverage:

```
ssh -i keys/id_ed25519 -p 2222 analyst@127.0.0.1 '
sudo python3 - <<"PY"
p="/var/ossec/etc/ossec.conf"
t=open(p).read()
line="    <directories realtime=\"yes\" report_changes=\"yes\">/home</directories>\n"
if ">/home<" not in t:
    t=t.replace("  </syscheck>", line+"  </syscheck>", 1)
    open(p,"w").write(t)
PY
sudo systemctl restart wazuh-agent'
```

This is what makes rule 100120 fire within a second of the key write.

---

3. Part C — Custom rule

The manager mounts the repository's `rules/` directory:

```
volumes:
  - /home/luqman/Documents/Eterna/soc-l1-lab/rules:/var/ossec/etc/rules:ro
```

Mount the **directory**, not a single file. A single-file bind mount follows the inode, and editors that replace the file (write-and-rename) leave the container reading the old copy. This cost an hour during development.

Deploy:

```
cd "$HOME/wazuh-docker/single-node"
docker compose up -d wazuh.manager
```

Verify the rule is visible:

```
docker exec single-node-wazuh.manager-1 \
  grep -c 100120 /var/ossec/etc/rules/local_rules.xml    # 2
```

Test it:

```
ssh -i "$LAB/lab/vm/keys/id_ed25519" -p 2222 analyst@127.0.0.1 \
  'sudo -u svc-backup bash -c "echo ssh-ed25519 AAAATEST >> /home/svc-backup/.ssh/authorized_keys"'
sleep 10
docker exec single-node-wazuh.manager-1 \
  grep '"id":"100120"' /var/ossec/logs/alerts/alerts.json | tail -1
```

Expected: one JSON object with `syscheck.path` and `sha256_after`.

---

4. Part D — Network sensor (optional for INC-001)

INC-001 is an SSH incident; Suricata cannot see encrypted guesses. The sensor matters for scenario 5 (scan) and scenario 3 (web). If you want the full lab:

```
ssh -i keys/id_ed25519 -p 2222 analyst@127.0.0.1 '
sudo apt-get install -y -t bookworm-backports suricata suricata-update
sudo sed -i "s|HOME_NET: .*|HOME_NET: \"[10.0.2.15/32]\"|" /etc/suricata/suricata.yaml
sudo suricata -T -c /etc/suricata/suricata.yaml
sudo systemctl restart suricata'
```

Rule narrowing is in `lab/suricata/README.md`. Keep it around 8,000 rules; the full 53k set will exhaust the VM.

---

5. Part E — Baseline snapshot

Take the snapshot **after** the agent is enrolled and **before** the attack:

```
cd "$LAB/lab/vm"
./snapshot.sh save clean-baseline
python3 - <<'PY'
import socket, time
s=socket.socket(socket.AF_UNIX, socket.SOCK_STREAM); s.settimeout(10)
s.connect("monitor.sock"); time.sleep(0.3); s.recv(65536)
s.sendall(b"info snapshots\n"); time.sleep(1)
print(s.recv(65536).decode(errors="replace"))
PY
```

Expected:

```
clean-baseline   1.37 GiB
```

Revert before each repeat:

```
./snapshot.sh load clean-baseline
```

---

6. Part F — Run the incident

Follow `04-attack-setup.md`. It creates the account, starts background noise, runs the brute force, and performs the post-exploitation.

Return here when the attack is complete.

---

7. Part G — Verify and extract

7.1 Confirm the agent is still active

```
docker exec single-node-wazuh.manager-1 /var/ossec/bin/agent_control -l | grep 001
```

7.2 Check each detection

```
for id in 5763 40112 100120; do
  echo "== $id =="
  docker exec single-node-wazuh.manager-1 \
    grep "\"id\":\"$id\"" /var/ossec/logs/alerts/alerts.json | tail -1 \
    | jq -r '[.timestamp,.rule.id,.rule.description] | @tsv'
done
```

Expected:

```
5763    sshd: brute force trying to get access to the system
40112   Multiple authentication failures followed by a success
100120  FIM: SSH authorized_keys added or modified
```

7.3 Export the window

```
docker exec single-node-wazuh.manager-1 cat /var/ossec/logs/alerts/alerts.json \
  > /tmp/alerts_all.json

python3 - <<'PY'
import json, datetime
start=datetime.datetime.fromisoformat("2026-10-30T15:21:00+00:00")
end=datetime.datetime.fromisoformat("2026-10-30T15:23:30+00:00")
rows=[]
for line in open('/tmp/alerts_all.json'):
```

```
    try: a=json.loads(line)
    except: continue
    if a.get('agent',{}).get('name')=='soc-victim' and \
       start <= datetime.datetime.fromisoformat(a['timestamp']) <= end:
        rows.append(a)
json.dump(rows, open('docs/evidence/INC-001-alerts.json','w'), indent=1)
print(len(rows), "alerts")
PY
```

7.4 Pull the raw logs

```
ssh -i lab/vm/keys/id_ed25519 -p 2222 analyst@127.0.0.1 \
  'sudo journalctl --since "2026-10-30 15:21:40" --until "2026-10-30 15:22:30" \
   _COMM=sshd --no-pager' > docs/evidence/INC-001-auth.log

ssh -i lab/vm/keys/id_ed25519 -p 2222 analyst@127.0.0.1 \
  'sudo tail -4000 /var/log/audit/audit.log' > /tmp/audit_raw.txt
```

Decode the session's commands:

```
python3 - <<'PY'
import re, datetime, binascii
start=int(datetime.datetime(2026,10,30,15,22,20,tzinfo=datetime.timezone.utc).timestamp())
end=int(datetime.datetime(2026,10,30,15,22,26,tzinfo=datetime.timezone.utc).timestamp())
groups={}
for ln in open('/tmp/audit_raw.txt', errors='replace'):
    m=re.search(r'audit\((\d+)\.\d+:(\d+)\)', ln)
    if m and start <= int(m.group(1)) <= end:
        groups.setdefault(m.group(2), []).append((int(m.group(1)), ln))
def dec(v):
    v=v.strip('"')
    if re.fullmatch(r'[0-9A-Fa-f]+', v) and len(v)%2==0 and len(v)>8:
        try:
            b=binascii.unhexlify(v)
            if all(32<=c<127 for c in b): return b.decode()
        except Exception: pass
    return v
out=[]
for _, recs in sorted(groups.items(), key=lambda kv: kv[1][0][0]):
    sysc=next((r for _,r in recs if 'type=SYSCALL' in r), '')
    ex=next((r for _,r in recs if 'type=EXECVE' in r), '')
    if 'uid=1002' in sysc and ex:
        args=dict(re.findall(r'(a\d+)="?([^" ]+)"?', ex))
        argv=[dec(args[k]) for k in sorted(args, key=lambda x:int(x[1:]))]
        out.append(f"{datetime.datetime.fromtimestamp(recs[0][0], datetime.timezone.utc):%H:%M:%S}Z
{' '.join(argv)}")
open('docs/evidence/INC-001-postex-commands.txt','w').write('\n'.join(out)+'\n')
print('\n'.join(out))
PY
```

8. Part H — Clean reset

Between runs:

```
# revert the VM
cd "$LAB/lab/vm" && ./snapshot.sh load clean-baseline

# remove the exercise account's key (if not reverted)
ssh -i keys/id_ed25519 -p 2222 analyst@127.0.0.1 \
  'sudo rm -f /home/svc-backup/.ssh/authorized_keys'

# keep the alerts; they are the evidence
```

Do **not** delete the Wazuh volumes if you want the evidence history. If you do, the dashboard starts empty.

9. Troubleshooting

Symptom	Cause	Fix
`permission denied` on Docker socket	User not in `docker` group	`sudo usermod -aG docker $USER`, re-login, or use a socket ACL
`newgrp: failed to read password`	User is not a member yet	Run the `usermod` first
Cloud-init exit 100	`suricata` is in `bookworm-backports` and apt will not cross suites	`apt-get install -t bookworm-backports suricata suricata-update`
Suricata OOM-kills the agent	Full ET Open (53k rules) in a 1.5 GB VM	Narrow by group; see `lab/suricata/README.md`
Suricata loads 0 rules	ET renamed the web rule files	`tar tzf` the cache and use the current names
No audit alerts	Audit keys do not match Wazuh's CDB list	Use `audit-wazuh-w` and `audit-wazuh-c`
Rule edit does not take effect	Single-file bind mount follows the old inode	Mount the `rules/` directory and recreate the manager
`<same_user />` never matches	Failures use `srcuser`, successes use `dstuser`	Do not rely on it; tune 40112 instead
`localhost` refused in Chrome	Resolves to `::1`, dashboard bound to IPv4 only	Add `[::1]:443:5601`
Certificate warning	Self-signed cert, SAN `wazuh.dashboard` only	Advanced → Proceed (or serve HTTP locally)
Dashboard 000 immediately after recreate	It takes ~30 s to boot	Wait and retry
VM `eth0` missing	Debian cloud images use predictable names	Use `ens3`

10. Reproduction checklist

- [ ] `docker compose ps` shows three healthy containers
- [ ] Dashboard returns `302`, manager API `200`
- [ ] Agent `001` is `Active`
- [ ] `auditctl -l` shows `audit-wazuh-c` and `audit-wazuh-w`
- [ ] `/home` is in the syscheck realtime directories
- [ ] Rule 100120 is present in the container
- [ ] Snapshot `clean-baseline` exists
- [ ] A test key write produces 100120
- [ ] The attack produces 5760 × 20, 5763, 40112, 100120

- [ ] Evidence files export without errors

---

11. Teardown

```
# stop the VM
pkill -f 'qemu-system.*soc-victim'

# stop the SIEM
cd "$HOME/wazuh-docker/single-node" && docker compose down

# remove data (destroys alerts)
docker compose down -v
```

The repository, rules, evidence, and write-ups are unaffected.